

Proyecto Corte 1

Materia: Frameworks y Herramientas para Big Data

Grupo: 2

Integrantes:

- Nicolás Santiago Cuarán Sotelo
- Sebastian Belalcazar Mosquera
- Bryan Fernando Burbano Carvajal
- Michel Dahiana Burgos Santos
- Juan David Daza Rivera

Fecha: 02/09/2026

Repositorio: <https://github.com/SEBASBELMOS/nyc-taxi-lakehouse>

1. Resumen ejecutivo

Se construyó un data stack completo sobre Docker Compose: ingesta de un Parquet público de NYC Yellow Taxi (enero 2025), aterrizaje en un data lake (MinIO), conversión a tabla Apache Iceberg catalogada en Nessie (vía Iceberg REST) y distribución final hacia Azure ADLS y ClickHouse. Toda la ingesta y carga se realiza con `dlt`; la lectura de Parquet se hace con PyArrow en lotes.

Resultado de la validación crítica: la tabla final en ClickHouse contiene exactamente **3.475.226 filas**, verificado de forma independiente en tres capas: RAW (MinIO), tabla Iceberg (vía Nessie) y ClickHouse. La verificación integral del stack cierra en **12/12 checks**, incluida la confirmación de los archivos cargados en Azure.

2. Entregables del enunciado

Entregable	Dónde está
4 scripts de ingesta y cargue de datos	<code>scripts/01..04_*.py</code> , ejecutados como notebooks en Jupyter (<code>notebooks/01..04_*.ipynb</code> , con outputs)
1 archivo <code>docker-compose.yml</code>	raíz del proyecto (4 servicios: <code>minio</code> , <code>nessie</code> , <code>clickhouse</code> , <code>jupyter</code>)
1 archivo con los secrets de <code>dlt</code>	<code>dlt/secrets.toml</code> (incluido en la carpeta de entrega; excluido del repositorio público)
Pantallazos solicitados	<code>evidencias/</code> (9 capturas, ver §7)
Servicios corriendo en Docker	evidencia 01
Dos buckets de MinIO con sus Parquet	evidencias 02, 03 y 03b
Catálogo de Nessie con el namespace	evidencia 04
Consulta con cantidad de registros (3475226)	evidencia 05
Consulta con tablas de metadatos en ClickHouse	evidencia 06
Consulta del archivo cargado en Azure	evidencia 07 (verificación programática con <code>adlfs</code>)

3. Arquitectura

Función	Tecnología	Servicio Docker
Data Lake / Object Storage	MinIO	<code>minio</code>
Catálogo	Nessie (Iceberg REST)	<code>nessie</code>
Data Warehouse	ClickHouse	<code>clickhouse</code>
Entorno interactivo	Jupyter	<code>jupyter</code>
File format	Apache Parquet	—
Table format	Apache Iceberg	—
Ingesta / carga	<code>dlt</code>	—
Lectura de Parquet	PyArrow	—



4. Configuración y decisiones técnicas

- **Secretos:** las credenciales de Azure, MinIO y ClickHouse viven en `dlt/secrets.toml` (excluido de control de versiones y montado como volumen de solo lectura en el contenedor de Jupyter). Ningún script ni notebook contiene credenciales, y ninguna aparece en los outputs guardados.
- **Entorno:** `.env` con los parámetros no secretos; `GROUP_FOLDER=GRUPO_2`.
- **Nessie como catálogo Iceberg REST:** Pylceberg no tiene un tipo de catálogo `nessie`; Nessie expone el protocolo Iceberg REST en `/iceberg` y el cliente se conecta con `RestCatalog`.
- **Catálogo persistente:** version store de Nessie en RocksDB sobre volumen (el default `IN_MEMORY` pierde namespace y tabla al reiniciar).
- **Memoria controlada:** la fuente (~3,5 M de filas) nunca se carga entera: `ParquetFile.iter_batches(batch_size=250.000)` y entrega de lotes PyArrow.
- **Puertos:** ClickHouse nativo se publica en el 9002 del host porque MinIO ocupa el 9000; dentro de la red Docker ClickHouse sigue en su 9000.
- **Nombres legibles:** `dataset_table_separator = "_"` en `dlt/config.toml`; la tabla final es `nyc_taxi__yellow_tripdata_2025_01` (el default de dlt).

usaría `__`).

- **Dependencias declaradas:** todas en `requirements.txt` e instaladas en la imagen (`Dockerfile`); no hay `pip install` dentro de los notebooks.

5. Ejecución de los pipelines

Los cuatro pipelines se ejecutaron **como notebooks dentro de Jupyter** (`notebooks/01..04`, con sus outputs guardados como evidencia). Las versiones `.py` equivalentes están en `scripts/` y pueden ejecutarse con `docker compose exec jupyter python scripts/<script>.py`; ambos formatos comparten la misma configuración (`scripts/config.py`).

5.1. Pipeline 01 — HTTP → MinIO RAW

- Fuente: `https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2025-01.parquet`
- Destino: `s3://nyc-taxi-raw/raw/yellow_tripdata/`
- Resultado: 1 archivo Parquet de 72,9 MB (`1788272855.525187.7eef0a56d0.parquet`).

```
Rows landed in MinIO: 3,475,226
OK - RAW bucket matches the expected row count.
```

5.2. Pipeline 02 — MinIO RAW → Iceberg + Nessie

- Tabla creada: `nyc_taxi.yellow_tripdata_2025_01` (20 columnas)
- Location: `s3://nyc-taxi-iceberg/nyc_taxi/yellow_tripdata_2025_01_25fa2203-bb5d-49f6-8162-bc570ef08446`
- Snapshots: 1 · Data files: 14 · Metadata files: 43
- Carga por lotes de 250.000 filas (`iter_batches` + `append`); antes de escribir se eliminan las columnas internas de dlt (`_dlt_load_id`, `_dlt_id`) para que la tabla Iceberg sea copia fiel del origen.

```
[OK] Iceberg row count matches - 3,475,226 (expected 3,475,226)
```

5.3. Pipeline 03 — Iceberg → Azure ADLS

- Origen: los data files se resuelven **preguntando al catálogo de Nessie** (`table.location()`), sin rutas hardcodeadas.
- Destino: `abfss://clase-4-dlt@fhbd.dfs.core.windows.net/GRUPO_2/nyc_taxi/`
- Resultado: data files Parquet (~73 MB) más la metadata de dlt, confirmados mediante verificación programática con `adlfs` (listado del contenedor con las mismas credenciales, sin depender del portal de Azure).

5.4. Pipeline 04 — Iceberg → ClickHouse

- Tabla final: `default.nyc_taxi_yellow_tripdata_2025_01`
- Tablas de metadatos de dlt creadas: `nyc_taxi_dlt_loads`, `nyc_taxi_dlt_pipeline_state`, `nyc_taxi_dlt_version`, `nyc_taxi_dlt_sentinel_table`.
- Carga completa en 3,02 s; `write_disposition="replace"` garantiza el conteo exacto en cada re-ejecución.

```
SELECT count(*) FROM default.nyc_taxi_yellow_tripdata_2025_01;
-> 3,475,226
[OK] ClickHouse row count matches - 3,475,226 (expected 3,475,226)
```

6. Verificación integral

`scripts/verificar.py` (también ejecutado como `notebooks/verificar.ipynb`)
recorre todo el stack en una sola corrida: **12/12 checks pasados**.

```
[OK ] bucket 'nyc-taxi-raw' exists
[OK ] bucket 'nyc-taxi-iceberg' exists
[OK ] RAW bucket has Parquet files - 1 file(s)
[OK ] Iceberg bucket has data files - 14 file(s)
[OK ] Iceberg bucket has metadata files - 43 file(s)
[OK ] namespace 'nyc_taxi' exists
[OK ] table 'nyc_taxi.yellow_tripdata_2025_01' registered
[OK ] Iceberg row count matches - 3,475,226 (expected 3,475,226)
[OK ] table 'nyc_taxi_yellow_tripdata_2025_01' exists
[OK ] dlt metadata tables present
```

```
[OK ] ClickHouse row count matches - 3,475,226 (expected 3,475,226)
[OK ] files present in Azure - 5 file(s)
```

7. Evidencias visuales

Las nueve capturas siguientes documentan cada punto solicitado por el enunciado. Los archivos originales están en la carpeta `evidencias/` del proyecto.

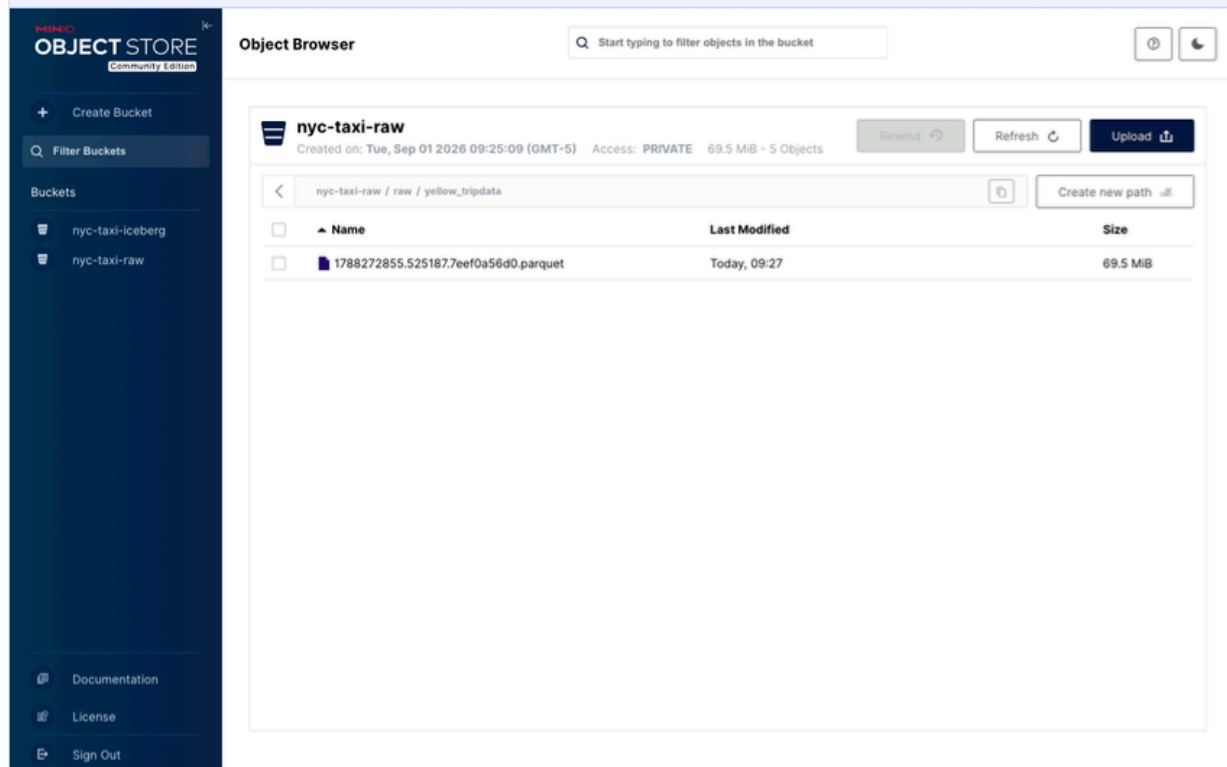
Figura 1 Servicios corriendo en Docker

```
C:\Users\sebas> docker ps
```

NAMES	IMAGE	STATUS	PORTS
cortel-jupyter	cortel-jupyter:1.0	Up (healthy)	0.0.0.0:8888->8888/tcp
cortel-nessie	ghcr.io/projectnessie/nessie:0.108.4	Up	0.0.0.0:19120->19120/tcp
cortel-clickhouse	clickhouse/clickhouse-server:25.8	Up (healthy)	0.0.0.0:8123->8123/tcp, 0.0.0.0:9002->9000/tcp
cortel-minio	quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z	Up (healthy)	0.0.0.0:9000-9001->9000-9001/tcp

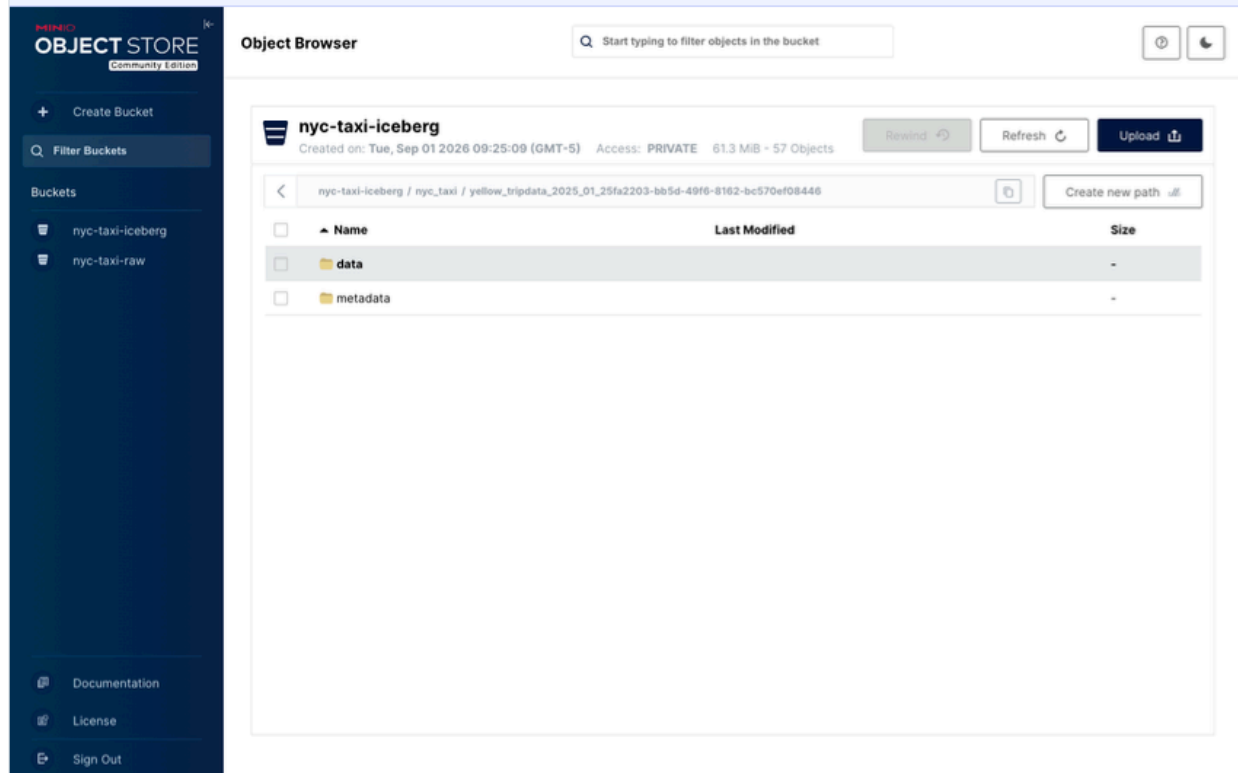
Los cuatro servicios del stack activos: `minio`, `nessie`, `clickhouse` y `jupyter`. El contenedor `minio-init` aparece finalizado (`Exited 0`) porque su única tarea es crear los buckets.

Figura 2 Bucket RAW en MinIO



Bucket `nyc-taxi-raw` con el archivo Parquet aterrizado por el pipeline 01 (72,9 MB, 3.475.226 filas).

Figura 3 Bucket Iceberg en MinIO



Bucket `nyc-taxi-iceberg` con la estructura de la tabla Apache Iceberg: carpeta `data/` (14 archivos Parquet) y `metadata/` (43 archivos).

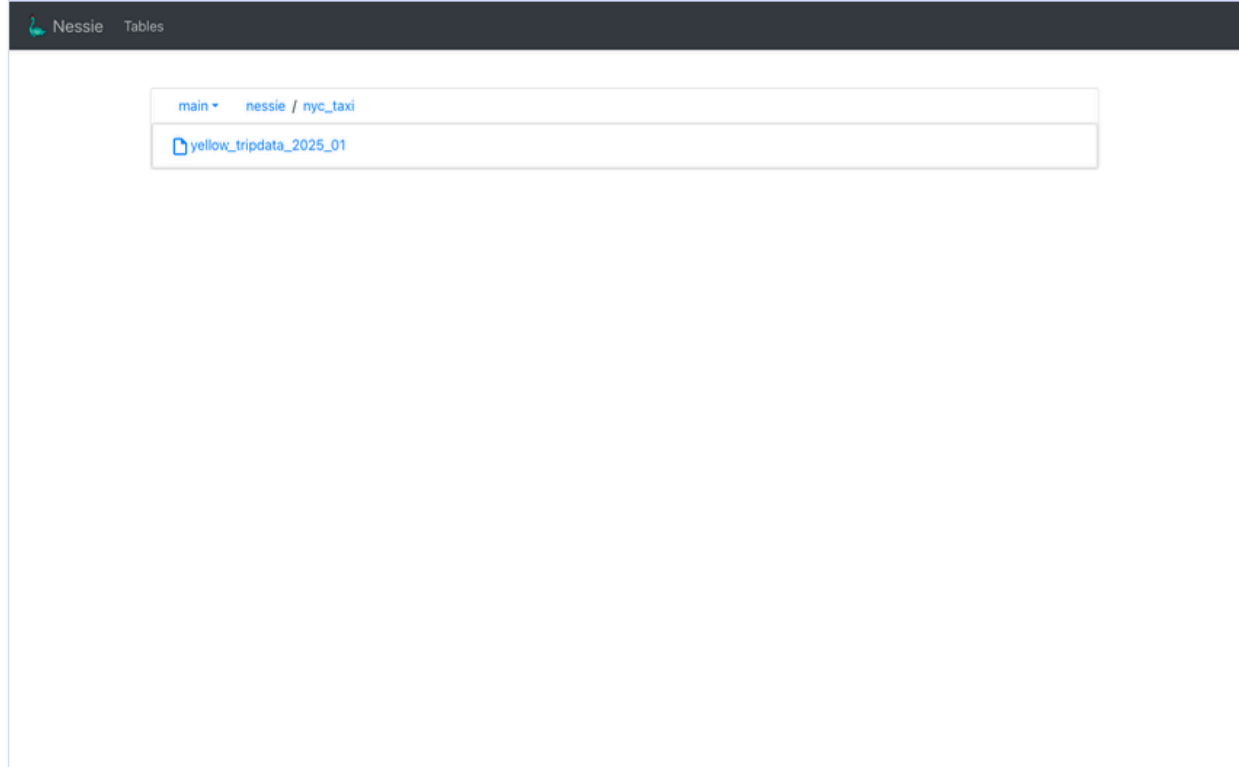
Figura 4 Metadata de la tabla Iceberg

The screenshot displays the 'Object Browser' interface for the 'nyc-taxi-iceberg' bucket. The sidebar on the left contains a 'Create Bucket' button, a 'Filter Buckets' search bar, and a list of buckets including 'nyc-taxi-iceberg' and 'nyc-taxi-raw'. The main content area shows the bucket's details: 'Created on: Tue, Sep 01 2026 09:25:09 (GMT-5)', 'Access: PRIVATE', '61.3 MiB', and '57 Objects'. Below this is a table of objects with columns for 'Name', 'Last Modified', and 'Size'. The table lists 15 metadata files, all with a size of 2.7 KiB and a last modified date of 'Today, 09:27'. The selected file is '00000-18519741-087b-49d1-a8c8-e569eb97de46.metadata...'.

Name	Last Modified	Size
00000-0f784819-7ce9-4344-a6c4-1ed9c9451472.metadata.js...	Today, 09:27	2.7 KiB
00000-18519741-087b-49d1-a8c8-e569eb97de46.metadata...	Today, 09:27	2.7 KiB
00000-292a6505-b906-497a-b89a-dd6266787505.metadata.j...	Today, 09:27	2.7 KiB
00000-2e7ca694-44e7-4cf3-b803-3eba8d61716e.metadata.js...	Today, 09:27	2.0 KiB
00000-356c7560-38e2-493f-8196-9c58f3e720a4.metadata.js...	Today, 09:27	2.7 KiB
00000-5e1d5459-c04f-493c-8acb-83632d8ed1c6.metadata.j...	Today, 09:27	2.7 KiB
00000-79a53edd-77a6-42df-baae-e773a8fabe26.metadata.json	Today, 09:27	2.7 KiB
00000-87668d80-e7cd-4fa5-a2f3-f1857c6f5dd5.metadata.json	Today, 09:27	2.7 KiB
00000-8b05f988-719c-470e-80c5-d7e1e67bdcf8.metadata.json	Today, 09:27	2.7 KiB
00000-9bb6cb94-2cf0-413f-a706-13a8e9d1cfed.metadata.json	Today, 09:27	2.7 KiB
00000-ce0c76e2-9e52-4e64-8d1a-b4e5d8e32816.metadata.j...	Today, 09:27	2.7 KiB
00000-e1abc0c8-c5a3-4d8a-a177-3eeabb93cc40.metadata.js...	Today, 09:27	2.7 KiB
00000-e329750f-c591-4ed8-8907-d73295342cd5.metadata.j...	Today, 09:27	2.7 KiB
00000-eb81f3b8-ea5f-4e9c-aadb-e8f29b32096b.metadata.js...	Today, 09:27	2.7 KiB

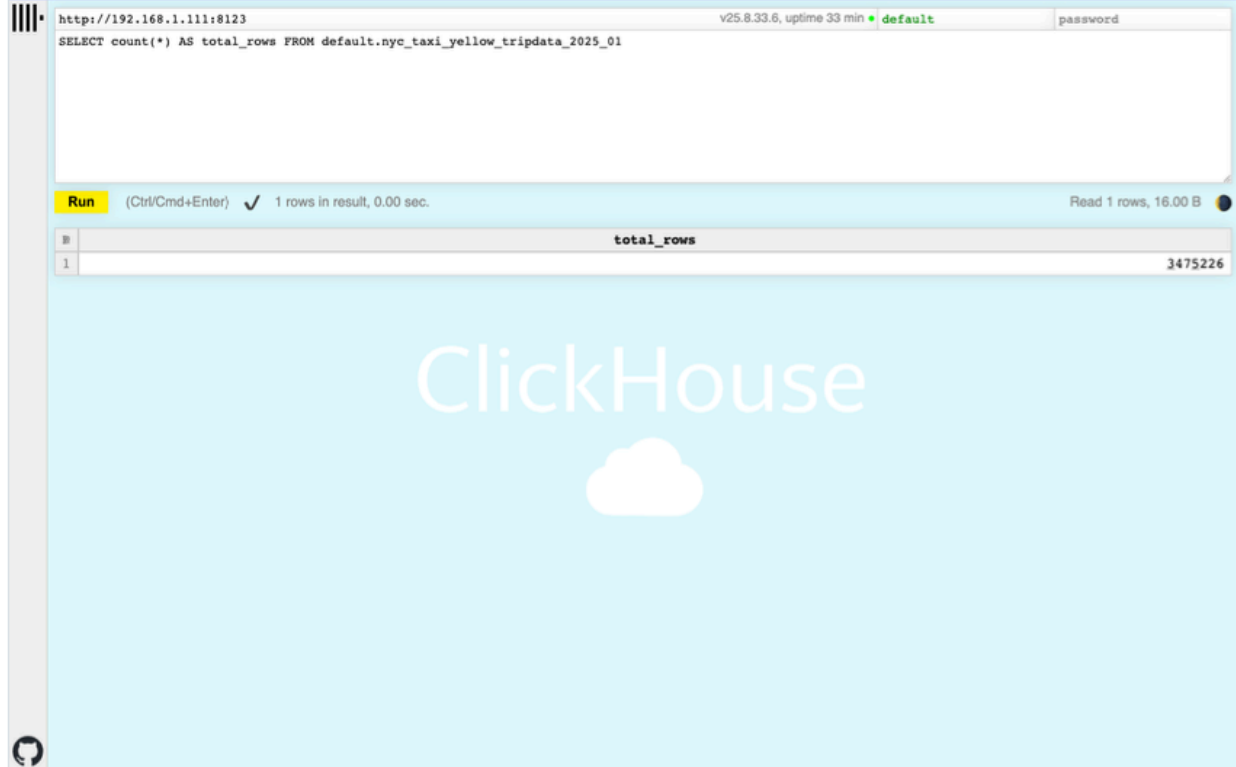
Detalle de los archivos `.metadata.json`, `.avro` y listas de manifiestos que Iceberg genera. Esta metadata es lo que convierte un conjunto de Parquet sueltos en una tabla con snapshots e historial.

Figura 5 Catálogo Nessie con el namespace



Namespace `nyc_taxi` y tabla `yellow_tripdata_2025_01` registrados en Nessie, con su *Metadata* Location apuntando al bucket Iceberg en MinIO.

Figura 6 Conteo de registros en ClickHouse

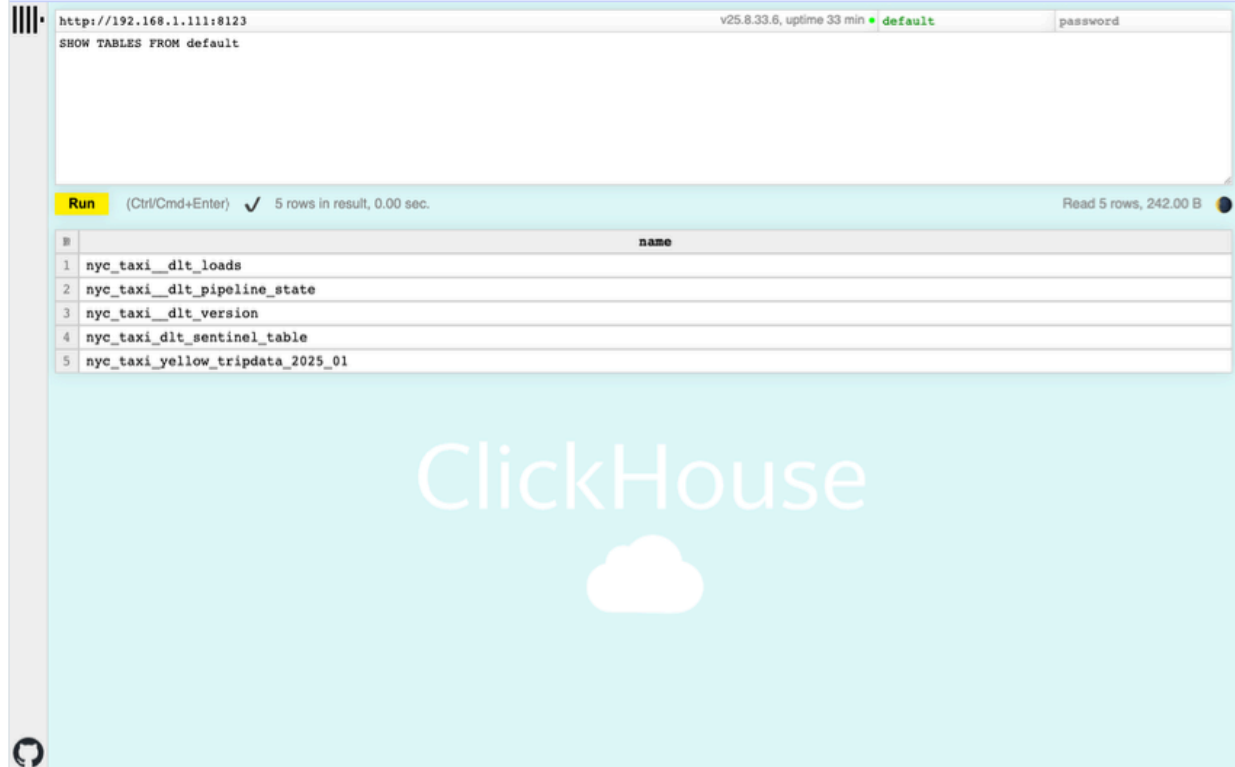


The screenshot displays the ClickHouse web interface. At the top, the URL is `http://192.168.1.111:8123` and the version is `v25.8.33.6`. The query input field contains the SQL statement: `SELECT count(*) AS total_rows FROM default.nyc_taxi_yellow_tripdata_2025_01`. Below the input, a yellow **Run** button is visible. The status bar indicates `1 rows in result, 0.00 sec.` and `Read 1 rows, 16.00 B`. The result is shown in a table with one row and one column named `total_rows`, containing the value `3475226`. The background features the ClickHouse logo and a cloud icon.

	total_rows
1	3475226

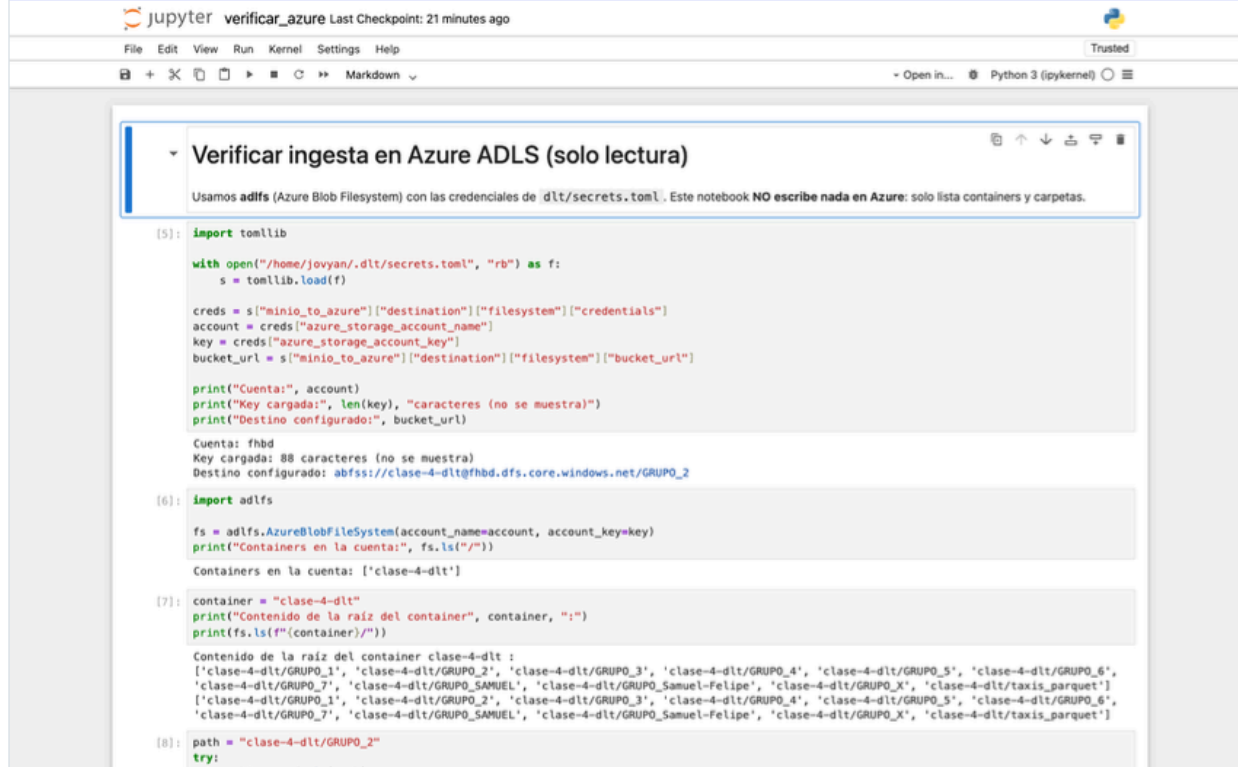
Resultado `SELECT count(*)` sobre la tabla final: 3.475.226 registros, exactamente el valor exigido por el de enunciado.

Figura 7 Tablas de metadatos en ClickHouse



Salida de `SHOW TABLES FROM default` : la tabla de datos junto a las tablas de metadatos que genera `dlt` (`_dlt_loads` , `_dlt_pipeline_state` , `_dlt_version` y la tabla centinela).

Figura 8 Archivo cargado en Azure ADLS



The screenshot shows a Jupyter Notebook interface with the title "Verificar ingesta en Azure ADLS (solo lectura)". Below the title, a text box states: "Usamos `adlfs` (Azure Blob Filesystem) con las credenciales de `dlt/secrets.toml`. Este notebook **NO** escribe nada en Azure: solo lista containers y carpetas."

The notebook contains three code cells:

```
[5]: import tomllib

with open("/home/jovyan/.dlt/secrets.toml", "rb") as f:
    s = tomllib.load(f)

creds = s["minio_to_azure"]["destination"]["filesystem"]["credentials"]
account = creds["azure_storage_account_name"]
key = creds["azure_storage_account_key"]
bucket_url = s["minio_to_azure"]["destination"]["filesystem"]["bucket_url"]

print("Cuenta:", account)
print("Key cargada:", len(key), "caracteres (no se muestra)")
print("Destino configurado:", bucket_url)

Cuenta: fhbd
Key cargada: 88 caracteres (no se muestra)
Destino configurado: abfss://clase-4-dlt@fhbd.dfs.core.windows.net/GRUPO_2

[6]: import adlfs

fs = adlfs.AzureBlobFileSystem(account_name=account, account_key=key)
print("Containers en la cuenta:", fs.ls("/") )

Containers en la cuenta: ['clase-4-dlt']

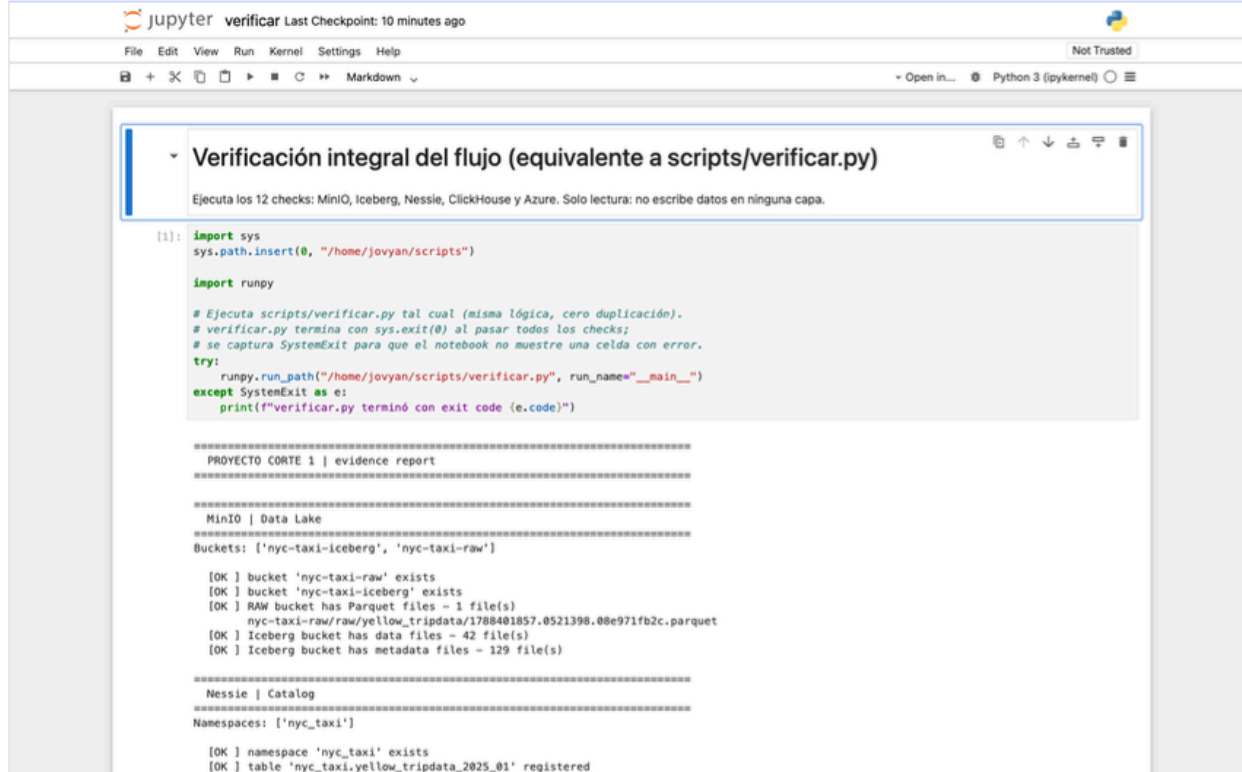
[7]: container = "clase-4-dlt"
print("Contenido de la raíz del container", container, ":")
print(fs.ls(f"{container}/"))

Contenido de la raíz del container clase-4-dlt :
['clase-4-dlt/GRUPO_1', 'clase-4-dlt/GRUPO_2', 'clase-4-dlt/GRUPO_3', 'clase-4-dlt/GRUPO_4', 'clase-4-dlt/GRUPO_5', 'clase-4-dlt/GRUPO_6',
'clase-4-dlt/GRUPO_7', 'clase-4-dlt/GRUPO_SAMUEL', 'clase-4-dlt/GRUPO_Samuel-Felipe', 'clase-4-dlt/GRUPO_X', 'clase-4-dlt/taxis_parquet']
['clase-4-dlt/GRUPO_1', 'clase-4-dlt/GRUPO_2', 'clase-4-dlt/GRUPO_3', 'clase-4-dlt/GRUPO_4', 'clase-4-dlt/GRUPO_5', 'clase-4-dlt/GRUPO_6',
'clase-4-dlt/GRUPO_7', 'clase-4-dlt/GRUPO_SAMUEL', 'clase-4-dlt/GRUPO_Samuel-Felipe', 'clase-4-dlt/GRUPO_X', 'clase-4-dlt/taxis_parquet']

[8]: path = "clase-4-dlt/GRUPO_2"
try:
```

Listado del contenedor `clase-4-dlt` mediante la librería `adlfs`, mostrando la carpeta `GRUPO_2/nyc_taxi/` con los data files y la metadata. La verificación es programática, sin depender del portal de Azure.

Figura 9 Verificación integral: 12/12 checks



```
[1]: import sys
    sys.path.insert(0, "/home/jovyan/scripts")

    import runpy

    # Ejecuta scripts/verificar.py tal cual (misma lógica, cero duplicación).
    # verificar.py termina con sys.exit(0) al pasar todos los checks;
    # se captura SystemExit para que el notebook no muestre una celda con error.
    try:
        runpy.run_path("/home/jovyan/scripts/verificar.py", run_name="__main__")
    except SystemExit as e:
        print(f"verificar.py terminó con exit code {e.code}")

=====
PROYECTO CORTE 1 | evidence report
=====

MinIO | Data Lake
=====
Buckets: ['nyc-taxi-iceberg', 'nyc-taxi-raw']

[OK ] bucket 'nyc-taxi-raw' exists
[OK ] bucket 'nyc-taxi-iceberg' exists
[OK ] RAW bucket has Parquet files - 1 file(s)
      nyc-taxi-raw/raw/yellow_tripdata/1788401857.0521398.08e971fb2c.parquet
[OK ] Iceberg bucket has data files - 42 file(s)
[OK ] Iceberg bucket has metadata files - 129 file(s)

=====
Nessie | Catalog
=====
Namespaces: ['nyc_taxi']

[OK ] namespace 'nyc_taxi' exists
[OK ] table 'nyc_taxi.yellow_tripdata_2025_01' registered
```

Ejecución de `verificar.py` recorriendo todo el stack en una sola corrida: buckets, archivos RAW, tabla Iceberg, catálogo Nessie, ClickHouse y Azure.

8. Incidentes y soluciones

Incidente	Causa	Solución
Nessie salía con <code>Permission denied</code> en <code>/nessie/data/LOG</code>	El volumen nombrado quedó con propietario root; la imagen corre como usuario <code>nessie</code> (uid 10000)	<code>chown -R 10000:10001</code> sobre el volumen, ejecutado con <code>--user 0</code>
<code>PermissionError</code> en <code>/home/jovyan/state/pipelines</code>	Volumen <code>jupyter-state</code> con propietario root; el contenedor corre como <code>jovyan</code> (uid 1000)	<code>chown -R 1000:1000</code> sobre el volumen
dlt rechazaba <code>secrets.toml</code> (<code>Empty key at line 1 col 0</code>)	El archivo se generó en UTF-8 con BOM	Regenerar el archivo en UTF-8 sin BOM

Incidente	Causa	Solución
	(PowerShell 5.1); el parser TOML no acepta BOM	
<code>docker compose up</code> fallaba con <code>logon session does not exist</code>	El CLI de Docker en sesión SSH no accede al credential store de la sesión interactiva de Windows	Ejecutar el <code>up</code> desde la sesión interactiva del usuario
Key de Azure con 115 caracteres en <code>secrets.toml</code>	Se concatenó texto sobrante a la key (una key válida de Azure mide 88 caracteres)	Regenerar <code>secrets.toml</code> desde la plantilla con la key del enunciado, verificando longitud y hash
Azure respondía <code>AccountIsDisabled</code>	La cuenta de almacenamiento <code>fhbd</code> estaba deshabilitada del lado del servicio	El profesor re-habilitó la cuenta; el pipeline 03 completó la carga

9. Reproducción desde cero

```
# dentro de la carpeta Proyecto-Corte-1-GRUPO_2 (incluye dlt/
secrets.toml y .env ya configurados)
docker compose up -d --build      # la primera build tarda unos minutos
docker ps                          # 4 contenedores Up (minio-init termina en Exited 0: solo crea los buckets)

# Ejecutar en orden los notebooks 01 → 02 → 03 → 04 en Jupyter
# <http://localhost:8888> (token: corte1)
# o los scripts equivalentes:
docker compose exec jupyter python scripts/01_http_to_minio.py
docker compose exec jupyter python scripts/02_parquet_to_iceberg.py
docker compose exec jupyter python scripts/03_minio_to_azure.
```

```
py
docker compose exec jupyter python scripts/04_minio_to_clickhouse.py
docker compose exec jupyter python scripts/verificar.py # 12/12 checks
```

Consolas: MinIO <http://localhost:9001> (admin/password123) · Nessie <http://localhost:19120> · ClickHouse <http://localhost:8123/play> (default/clickhouse123) · Jupyter <http://localhost:8888> (token `cortel`).
Los pipelines son idempotentes: re-ejecutarlos no duplica datos.

10. Conclusiones

El flujo queda completo de punta a punta: el dato público se ingiere con `dlt`, se persiste como Parquet en MinIO (data lake), se promueve a tabla Apache Iceberg con catálogo Nessie —separando file format, table format y catálogo como componentes independientes— y se distribuye a los dos destinos finales: Azure ADLS (carpeta `GRUPO_2`) y ClickHouse (data warehouse).

La validación crítica se cumple con exactitud: **3.475.226 filas** en las tres capas de datos, y la verificación integral cierra en **12/12 checks**, incluida la confirmación programática de los archivos en Azure. El stack es reproducible con un solo `docker compose up -d --build` y los secretos quedan fuera del código y del repositorio público en todo momento.